C++ Functions and Classes

Lesson 2 / Parameters and collaborating objects

What happens after a swap?

```
int x = 10, y = 20;
swap1(x, y);
std::cout << x << ' ' << y << '\n';
```

Keep your prediction. We will return to it.

Functions

How a call expression receives a result

The test_add interface

```
int test_add(int a, int b) {
    int sum = a + b;
    return sum;
}

int result = test_add(1, 2);
```

Parameters a and b receive 1 and 2. result receives 3.

Local state during one call

```
call                    a    b    sum
test_add(1, 2)          1    2      3
return 3              local objects end
caller                result = 3
```

The result value returns. Local variables do not become globals.

Output and return values

```cpp
int test_add(int a, int b) {
    std::cout << "adding\n";
    return a + b;
}

int result = test_add(2, 3);
```

The screen shows adding. result receives 5.

A return value can join another expression

```
int total = test_add(2, 3) * 10;
std::cout << total << '\n';
```

Receive 5, multiply by 10, then print 50.

Declarations, definitions, and void

```
int test_add(int a, int b);

void hello() {
    std::cout << "Hello\n";
}
```

A declaration gives the interface. A definition supplies the body.
void means no return value.

Local names do not share storage

```
void change(int value) {
    value = 99;
}

int value = 10;
change(value);
```

The same name in separate scopes still names separate objects.

References and pointers

Names, addresses, and the objects at those addresses

A reference aliases an object

```
std::string band1("poppin party");
std::string& live1 = band1;
live1 = "afterglow";
```

What does band1 contain now?

Assignment through a reference does not rebind it

```
int a = 10, b = 20;
int& r = a;
r = b;
b = 30;
```

Finally a=20, b=30, r=20.

A pointer stores an address

```
std::string* anime1 = &band1;

std::cout << *anime1 << '\n';
*anime1 = "roselia";
```

&band1 takes an address. *anime1 accesses the target object.

Declaration, address-of, and dereference

```
int value = 10;
int* p = &value;

// int*    : type of p
// &value  : address of value
// *p      : object at that address
```

Read the syntax context before interpreting the symbol.

Changing a pointer or its target

```
std::string a("A"), b{"B"};
std::string* p = &a;
p = &b;
*p = "X";
```

a is A. b is X.

p changes its stored address, then writes to the new target.

Null pointers and lifetimes

```
std::string* p = nullptr;
if (p != nullptr) {
    std::cout << *p << '\n';
}
```

Check for a target before dereferencing.

Read–only borrowing and lifetime

```
const std::string& view = band1;
const std::string* ptr = &band1;

// view = "X"; // not allowed
// *ptr = "X"; // not allowed
```

const restricts mutation through this access path.

The original object must remain alive while borrowed.

Practice: trace the objects

```
int a = 10, b = 20;
int& r = a;
int* p = &b;
r = 11;
*p = r;
p = &a;
```

What are the final values of a, b, r, and *p?

Line–by–line results of the trace

operation	a	b	r	*p
r = 11	11	20	11	20
*p = r	11	11	11	11
p = &a	11	11	11	11

The final line changes the pointer target, not a or b.

The swap experiment

Parameter types determine which data a function can access

swap1 exchanges local copies

```
void swap1(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

swap1(x, y);
```

The state trace of swap1

step	x	y	a	b	temp
call	10	20	10	20	–
temp=a	10	20	10	20	10
a=b	10	20	20	20	10
b=temp	10	20	20	10	10

No step writes to x or y.

swap2 writes through addresses

```
void swap2(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

swap2(&x, &y);
```

Parameters copy addresses. *a and *b access the originals.

The state trace of swap2

step	target of a	target of b
call	x=10	y=20
temp=*a	x=10	y=20
*a=*b	x=20	y=20
*b=temp	x=20	y=10

The pointers are copies, but their targets are the caller’s objects.

Swapping with reference parameters

```
void swap_ref(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

swap_ref(x, y);
```

a and b directly alias x and y.

Choosing a parameter interface

```
void f(int value);          // independent copy
void f(int& value);         // may modify caller
void f(const int& value); // read-only borrow
void f(int* value);         // pointer value
```

An interface should clarify mutation and whether no target is allowed.

Practice: repair a function that cannot write back

```
void add_bonus(int score) {
    score += 5;
}
```

After add_bonus(score), the caller’s score must increase by 5.
What needs to change?

Mutation and returning a new value

```
void add_bonus(int& score) {
    score += 5;
}

int with_bonus(int score) {
    return score + 5;
}
```

add_bonus(score) mutates directly.

score = with_bonus(score) explicitly stores a new value.

Break

10 minutes

Next: extend Band and trace the integrated program

Band: data and behavior

Each band has its own members and operations

From a record to Band

```
class Band {
private:
    std::string keyboard;
public:
    void set_keyboard_anon() {
        keyboard = "anon";
    }
};
```

Public methods change internal state.

Five fields belong to one band

```
guitar    mutsumi
vocal     hatsune
drum      nyamu
bass      umiri
keyboard  saki
```

Field names belong to the type. Each object holds its own values.

Objects have independent state

```
Band a;
Band b;
a.set_keyboard_anon();
```

Does changing a automatically change b?

Independent state in two objects

```
before          a.keyboard=""   b.keyboard=""
a.set_keyboard_anon()
after           a.keyboard="anon"
                b.keyboard=""
```

Matching member names do not imply shared storage.

private and public

```
a.keyboard = "anon";  // error
a.set_keyboard_anon(); // OK
```

Member functions can access their class's private members.

Encapsulation provides one update interface

```
void set_keyboard(const std::string& name) {
    if (name.empty()) return;
    keyboard = name;
}
```

The caller requests a change. The class decides whether to accept it.

Constructors establish initial state

```
Band(std::string key) {
    keyboard = key;
}

Band a("saki");
```

The constructor has the class name and no return type.

Parameterized and default construction

```
Band a("saki"); // OK in the reduced class
// Band b;      // no matching constructor
```

Declaring this constructor prevents an implicit default constructor.

Member initializer lists

```
Band(std::string key)
    : keyboard(key) {}
```

keyboard is initialized directly from key.

Members initialize in declaration order.

Declaration order controls initialization

```
class Example {
    int first;
    int second;
public:
    Example() : second(2), first(1) {}
};
```

first still initializes before second.

Writing the list in declaration order makes this clear.

Reading and changing state

```
void print_info() const {
    std::cout << keyboard << '\n';
}

void set_keyboard_anon() {
    keyboard = "anon";
}
```

The trailing const prevents changes to ordinary members.

What can a const object call?

```
const Band band("saki");
band.print_info();        // OK
// band.set_keyboard_anon(); // error
```

Read-only access becomes part of the callable interface.

Practice: complete a small Band

Keep one private string: keyboard

Accept its initial value in a constructor

Provide print and update methods

Create two objects and change only one

Reviewing the Band exercise

a starts as saki, b as eve

After changing only a: anon / eve

A const object can print

Direct keyboard access outside the class fails to compile

The five members of the full Band

```
Band ave_mujica(
    "mutsumi", "hatsune", "nyamu",
    "umiri", "saki"
);
ave_mujica.print_info();
ave_mujica.live();
```

guitar, vocal, drum, bass, keyboard

A vector of bands

```
std::vector<Band> live_bands;
live_bands.emplace_back(
    "hina", "aya", "maya", "chisato", "eve");
live_bands.emplace_back(
    "sayo", "yukina", "ako", "lisa", "rinko");
```

vector manages the collection. Band defines each object.

Vector size and element access

```
std::cout << live_bands.size() << '\n';
live_bands[0].print_info();
live_bands.at(1).print_info();
```

After two appends, size is 2 and valid indices are 0 and 1.

Which object does a loop change?

```
for (Band band : live_bands) {
    band.set_keyboard_anon();
}

for (Band& band : live_bands) {
    band.set_keyboard_anon();
}
```

The first loop changes copies. The second changes stored objects.

Read-only iteration over bands

```
for (const Band& band : live_bands) {
    band.print_info();
}
```

Borrow each stored object and read through its const interface.

Classes and containers checkpoint

What does a value, reference, or pointer parameter provide?

When is an object’s initial state established?

Why can a read–only method be const?

Inheritance

Extend Band with vocal_actor

MofoBand inherits Band

```
class MofoBand : public Band {
protected:
    std::string vocal_actor;
public:
    void print_mofo_info() const {
        print_info();
        std::cout << vocal_actor << '\n';
    }
};
```

Reuse Band behavior and add the voice actor.

The base initializes first

```
MofoBand(std::string gt, std::string vo,
         std::string dr, std::string ba,
         std::string key, std::string actor)
    : Band(gt, vo, dr, ba, key),
      vocal_actor(actor) {}
```

Band first, then vocal_actor, then the constructor body.

Access to a protected member

MofoBand member functions: allowed

MofoBand_New member functions: allowed under inheritance rules

Direct object.vocal_actor in main: not allowed

this points to the current object

```
void change_actor(
    const std::string& new_actor) {
    this->vocal_actor = new_actor;
}
```

The left side is a member of this object. The right side is a parameter.

Practice: change the actor

Construct an object with actor amane

Call change_actor("ayane")

Print the updated information

Explain why main cannot directly change the protected field

Tracing class_band

ave_mujica: construct, print_info, live

live_bands: append two bands, iterate

ave_mujica: get_guitar, change keyboard

morfonica and lovelive: construct, change actor

Core lesson review

Which object owns this data?

Does the function access a copy or the original?

Which interface lets external code change it?

Optional extensions

Virtual functions, slicing, and ownership

Calling derived behavior through a base

```
struct Base {
    virtual void live() const { /* ... */ }
    virtual ~Base() = default;
};
struct Derived : Base {
    void live() const override { /* ... */ }
};
```

A Base& referring to Derived dispatches live to Derived.

Copying by value can slice an object

```
Derived d;
Base copied = d;
Base& borrowed = d;
```

copied contains only a Base object.

borrowed still refers to the complete d.

Exclusive resource ownership

```
std::unique_ptr<Band> owner = std::make_unique<Band>(
    "mutsumi", "hatsune", "nyamu",
    "umiri", "saki");
std::unique_ptr<Band> next = std::move(owner);
```

next receives ownership. owner becomes empty.

The resource is released automatically at scope exit.

After class and challenge work

Required: Assessment–02 using the lesson interfaces

Extra practice: choose weak topics from the exercise index

Challenges: read the relevant extension material first